

Data Wrangling 🤠

Dictionaries

Lesson 13

Our Current Tool

- Lists store values by **position** — useful when order matters
- But what if we want to look something up by a **meaningful name**?

```
names = ["Alicia", "Leon", "Greg"]  
numbers = ["555-0142", "555-0198", "555-0173"]  
# which index is Greg's number?
```

Dictionaries

- A **dictionary** stores **key-value pairs**
- Look up a value by its **key** instead of an index
- Keys are typically strings; values can be any type

```
phone_book = {"Alicia": "555-0142", "Leon": "555-0198", "Greg": "555-0173"}
```

Creating a Dictionary

- **Empty dictionary:** add entries later

```
phone_book = {}
```

- **Dictionary literal:** entries specified at initialization

```
phone_book = {"Alice": "555-0142", "Leon": "555-0198", "Greg": "555-0173"}  
grades = {"Alice": 91, "Leon": 84, "Greg": 78}
```

- Each entry is written as `key: value`, separated by commas
- Keys must be **unique**

Demo: Accessing Values

- Use square brackets with the **key** to read a value

```
>>> grades = {"Alice": 91, "Leon": 84, "Greg": 78}
>>> grades["Alice"]
91
>>> grades["Leon"]
84
>>> grades["Diana"]
KeyError: 'Diana'
```

Demo: Safe Access with `.get()`

- `.get()` returns `None` instead of raising a `KeyError`

```
>>> grades.get("Diana")
>>> grades.get("Diana", 0)
0
>>> grades.get("Alice", 0)
91
```

Demo: Updating Values

- Assign to an existing key to change its value

```
>>> grades = {"Alice": 91, "Leon": 84, "Greg": 78}
>>> grades["Leon"] = 90
>>> grades
{"Alice": 91, "Leon": 90, "Greg": 78}
```

Demo: Adding New Entries

- Assign to a **new key** to add an entry

```
>>> grades["Diana"] = 88
>>> grades
{"Alice": 91, "Leon": 90, "Greg": 78, "Diana": 88}
>>> grades["Eve"] = 73
>>> grades
{"Alice": 91, "Leon": 90, "Greg": 78, "Diana": 88, "Eve": 73}
```


Demo: Removing Entries

- `pop` removes an entry by key and returns its value

```
>>> grades.pop("Greg")
78
>>> grades
{"Alice": 91, "Leon": 90, "Diana": 88, "Eve": 73}
>>> grades.pop("Zara", None)  # safe pop: no KeyError
None
```

Checking Membership

- Use `in` to check whether a **key** exists

```
>>> "Alice" in grades
True
>>> "Zara" in grades
False
>>> 91 in grades
False
```

Dictionary Operations

Operation	Form	Result
Length	<code>len(d)</code>	number of key-value pairs
Access	<code>d[key]</code>	value at key (KeyError if missing)
Safe access	<code>d.get(key, default)</code>	value or default

Operations Cont'd

Operation	Form	Result
Update	<code>d[key] = value</code>	updates existing entry
Add	<code>d[new_key] = value</code>	adds new entry
Remove	<code>d.pop(key)</code>	removes entry, returns value
Membership	<code>key in d</code>	<code>True</code> if key exists

Demo: Iterating Over a Dictionary

- Loop over **keys** with a `for` loop

```
for name in grades:  
    print(f"{name}: {grades[name]}")
```

- Or loop over **key-value pairs** with `.items()`

```
for name, score in grades.items():  
    print(f"{name}: {score}")
```

Demo: Class Average

- Use a dictionary to compute a summary statistic

```
# class_average.py
grades = {"Cornelius": 91, "Ivana": 90, "Vanderbilt": 88, "Alicia": 73}

total = 0
for name in grades:
    score = grades[name]
    total = total + score

average = total / len(grades)
print(average)
```

Demo: Finding Durham-Area Contacts

- Filter a dictionary based on a condition on its values

```
# phone_book.py
phone_book = {
    "Liam": "919-555-0142",
    "Yash": "704-555-0198",
    "Seraphina": "919-555-0173",
    "Aarna": "336-555-0184",
}
```

```
durham_contacts = []  
for name in phone_book():  
    number = phone_book[name]  
    area_code = number.split("-")[0]  
    if area_code == "919":  
        durham_contacts.append(name)  
  
print(durham_contacts)
```


Lists vs. Dictionaries

	<code>list</code>	<code>dict</code>
Access by	index (position)	key (label)
Order	yes	insertion order
Duplicates	yes	keys must be unique
Use when	sequence of values	labeled values